EE8 CPU

Instruction Set Architecture Specification

Version 4.1

Second Year Electrical Engineering

Engineering Design Module (EG2300)

The EE8 is a simple 8-bit CPU designed for educational purposes.
Students will implement this CPU in Logisim to control a batch diverter manufacturing station.

Contents

1 Overview **3**
1.1 Key Specifications . . . 3

2 Registers **3**
2.1 R3 and Comparison Operations . . . 3

3 Platform Signals & Safety Logic **4**
3.1 Signal Definitions . . . 4
3.2 REBOOT . . . 4
3.3 STOP . . . 4
3.4 FAULT . . . 5
3.5 SYS_ERR . . . 5
3.6 Platform Halt Logic . . . 5
3.7 Alarm Logic . . . 5

4 I/O Interface **6**
4.1 Port Definitions . . . 6
4.2 ITEM_PULSE Semantics . . . 6
4.3 Reset State . . . 6
4.4 Option A: Memory-Mapped I/O . . . 6
4.5 Option B: Dedicated I/O Instructions . . . 7
4.6 Option C: Register-Mapped I/O . . . 7

5 Instruction Formats **7**
5.1 R-Type (Register Operations) . . . 7
5.2 I-Type (Immediate) . . . 7
5.3 J-Type (Jump) . . . 8

6 Instruction Set Reference **8**
6.1 Arithmetic Instructions . . . 8
6.1.1 ADD — Add . . . 8
6.1.2 SUB — Subtract . . . 9
6.2 Bitwise Instructions . . . 9
6.2.1 AND — Bitwise AND . . . 9
6.2.2 OR — Bitwise OR . . . 9
6.2.3 XOR — Bitwise Exclusive OR . . . 10
6.3 Shift Instructions . . . 10
6.3.1 SHL — Shift Left . . . 10
6.3.2 SHR — Shift Right . . . 10
6.4 Data Movement Instructions . . . 10
6.4.1 MOV — Move (Copy Register) . . . 10
6.4.2 LDI — Load Immediate . . . 11
6.5 Comparison Instructions . . . 11
6.5.1 LT — Less Than . . . 11
6.5.2 EQ — Equal . . . 11
6.6 Control Flow Instructions . . . 12
6.6.1 JMP — Unconditional Jump . . . 12
6.6.2 JZ — Jump if Zero . . . 12
6.6.3 JNZ — Jump if Not Zero . . . 12

6.7 Miscellaneous Instructions . . . 13
6.7.1 Reserved — Opcode 1110 . . . 13
6.7.2 HALT — Halt Execution . . . 13
6.8 I/O Instructions (Option B) . . . 13
6.8.1 IN — Read Input Port . . . 14
6.8.2 OUT — Write Output Port . . . 14
6.8.3 I/O Encoding Summary (Option B) . . . 14

7 Instruction Encoding Summary **15**

8 Binary Encoding Examples **15**
8.1 Example 1: ADD R0 R1 R2 . . . 15
8.2 Example 2: LDI R0 42 . . . 15
8.3 Example 3: JNZ R3 5 . . . 16
8.4 Example 4: JZ R0 3 . . . 16
8.5 Example 5: JMP 10 . . . 16
8.6 Example 6: LT R0 R1 (compare) . . . 16
8.7 Example 7: IN R0 (Option B) . . . 16
8.8 Example 8: OUT R2 (Option B) . . . 16

9 Program Memory **17**
9.1 Program Counter (PC) . . . 17
9.2 Loading Programs . . . 17
9.3 Fetch-Decode-Execute Cycle . . . 17

10 Timing and Clocking **17**
10.1 Clock Phases (single-cycle) . . . 17

11 Reset State Summary **18**

12 Assembly Language Syntax **18**
12.1 Comments . . . 18
12.2 Labels . . . 18
12.3 Numeric Formats . . . 18

13 Example Programs **19**
13.1 Batch Diverter: $A \neq B$ with Full Alternation (Option B) . . . 19
13.2 Simple Batch Diverter: Equal Counts (Option B) . . . 20
13.3 Arithmetic Example: Add Two Numbers . . . 20
13.4 Find Maximum of Two Numbers . . . 20
13.5 Polling Input Example (Option B) . . . 21
13.6 Countdown Example . . . 21

A Quick Reference Card **22**

B Opcode Map **22**

C I/O Interface Options Summary **23**

D Changes from Previous Versions **23**

1 Overview

The EE8 is a simple 8-bit CPU designed for educational purposes. It features a 16-bit instruction word, 4 general-purpose registers, and a minimal RISC-style instruction set. Students will implement this CPU in Logisim to control a batch diverter manufacturing station.

1.1 Key Specifications

Parameter	Value
Data width	8 bits
Instruction width	16 bits
Number of registers	4
Addressing	4-bit (16 instructions max)
Program memory	16-word ROM
Signed representation	Two's complement

2 Registers

The EE8 has four 8-bit general-purpose registers, addressed with 2 bits.

Code	Name	Description
00	R0	General purpose register
01	R1	General purpose register
10	R2	General purpose register
11	R3	General purpose register (also used for comparison results)

All registers are fully general-purpose and can be used for computation, storage, or I/O operations.

2.1 R3 and Comparison Operations

R3 serves double duty as the destination for comparison instructions:

- **LT Rs1 Rs2** sets R3 to 1 if Rs1 $<$ Rs2, otherwise 0
- **EQ Rs1 Rs2** sets R3 to 1 if Rs1 $=$ Rs2, otherwise 0
- R3 can be read and written like any register, but comparison instructions will overwrite it

Design Note

Unlike traditional CPUs (x86, ARM) which use packed bit flags (N, Z, C, V), EE8 uses a full 8-bit register for comparison results. This design choice follows modern RISC philosophy (RISC-V):

Advantages:

- **Simpler hardware** — No bit-field extraction logic needed
- **Orthogonal design** — `R3` can be used like any other register
- **Easier to implement** — Students build standard 8-bit register, not special flag register
- **Modern approach** — RISC-V (2010s) uses explicit comparison results, not implicit flags

Trade-offs:

- Cannot branch on overflow or unsigned comparisons without additional instructions
- Acceptable for EE8's educational scope and target applications

3 Platform Signals & Safety Logic

The batch diverter station has safety signals that are enforced in **hardware**, external to the CPU's instruction execution. The CPU does **not** read or poll these signals — they act on the platform directly.

3.1 Signal Definitions

Signal	Type	Description
REBOOT	Input (async)	System reset. Clears all CPU state and restarts execution.
STOP	Input	Operator halt/pause request.
FAULT	Input	Non-emergency fault (jam, guard open, etc.).
SYS_ERR	Internal (latched)	Illegal instruction detected. Set by decode logic.
CPU_HALT	Internal (latched)	Set by the **`HALT`** instruction.

3.2 REBOOT

REBOOT is an asynchronous hardware reset signal.

While REBOOT = 1:

- PC is forced to 0
- All registers are cleared to 0
- Output port is cleared to 0 (MOTOR_EN=0, GATE_SEL=0)
- SYS_ERR latch is cleared
- CPU_HALT latch is cleared

Execution starts from address 0 when REBOOT returns to 0. REBOOT is the **only** way to clear SYS_ERR and CPU_HALT.

3.3 STOP

STOP is a hardware halt/pause signal (operator-initiated).

While STOP = 1:

- CPU clock is gated — PC is frozen, no instructions execute
- All registers and output port are preserved (retain last values)
- Execution resumes from the paused point when STOP returns to 0

3.4 FAULT

FAULT is a hardware halt/pause signal with alarm (fault condition).

While FAULT = 1:

- CPU clock is gated — same freeze behaviour as STOP
- ALARM output is asserted (see Section 3.7)
- Execution resumes from the paused point when FAULT returns to 0

3.5 SYS_ERR

SYS_ERR is a latched internal signal set by the instruction decoder on illegal instruction detection.

Triggers:

- Execution of undefined opcode `1110` (when no team extension is implemented)
- Execution of undefined sub-encoding (`Rs2=01` in the MOV/IN/OUT family under Option B)

Behaviour when set:

- SYS_ERR is **latched** — once set, it remains asserted until REBOOT
- PC is frozen (no further instructions execute)
- MOTOR_EN is forced to 0 (see Section 3.6)
- ALARM is asserted (see Section 3.7)

Clearing: Only REBOOT clears the SYS_ERR latch.

3.6 Platform Halt Logic

The platform combines all halt sources into a single HALTED signal:

$$\text{HALTED} = \text{STOP} \mid \text{FAULT} \mid \text{SYS\_ERR} \mid \text{CPU\_HALT}$$

When HALTED is asserted, the conveyor motor is unconditionally disabled:

$$\text{MOTOR\_EN}_{\text{pin}} = \text{MOTOR\_EN}_{\text{cpu}} \ \& \ \overline{\text{HALTED}}$$

This means the CPU can *request* MOTOR_EN=1 via the output port, but the platform will override it to 0 whenever any halt condition is active. This is a hardware safety interlock.

3.7 Alarm Logic

The ALARM output is driven by hardware, not by the CPU's output port:

$$\text{ALARM} = \text{FAULT} \mid \text{SYS\_ERR}$$

ALARM is **not** asserted for STOP (operator-initiated, not a fault) or CPU_HALT (intentional program termination). ALARM is not part of the CPU's output port — teams do not need to set or clear it in software.

Important

These signals are **not** part of the ISA. The CPU does not execute instructions to read STOP, FAULT, or REBOOT. They are external hardware signals that act on the platform and clock/reset logic. Students implement these as combinational logic outside the CPU datapath.

4 I/O Interface

The EE8 interfaces with the batch diverter station through input and output ports. Teams must choose **one** of the following I/O strategies and document their choice in the report.

4.1 Port Definitions

Input Port:

Bit	Signal	Description
0	ITEM_PULSE	Item detection flag (see Section 4.2)

Bits 1–7 are reserved (read as 0).

Output Port:

Bit	Signal	Description
0	GATE_SEL	Active packing lane (0 = Gate A, 1 = Gate B)
1	MOTOR_EN	Conveyor motor enable request (see Section 3.6)
2+	STATUS	Debug/status bits (optional, team-defined)

Note: MOTOR_EN is a *request* — the platform overrides it to 0 when HALTED (see Section 3.6). ALARM is driven by hardware safety logic (see Section 3.7) and is not part of the CPU output port.

4.2 ITEM_PULSE Semantics

ITEM_PULSE is a **hardware-latched flag** with the following behaviour:

- **Set** by external hardware when an item passes the counting sensor
- **Cleared on read** — when the CPU reads the input port, the ITEM_PULSE latch is automatically reset to 0

This "clear-on-read" mechanism guarantees exactly one count per item. The CPU does not need to wait for the pulse to go low; each read either sees a 1 (item arrived since last read) or 0 (no item). This prevents multi-counting from polling too fast.

4.3 Reset State

On reset (REBOOT), the output port is cleared to 0:

- GATE_SEL = 0 (Gate A selected)
- MOTOR_EN = 0 (motor off)
- STATUS = 0

4.4 Option A: Memory-Mapped I/O

Specific addresses in the ROM address space are aliased to I/O ports instead of instructions. This approach requires additional decode logic but no new instructions.

Implementation:

- Address `0xE` (14): Reading fetches input port value instead of instruction
- Address `0xF` (15): Writing stores to output port instead of executing

4.5 Option B: Dedicated I/O Instructions

Add explicit `IN` and `OUT` instructions to the ISA. This is the cleanest approach for educational purposes.

New Instructions:

Opcode	Binary	Mnemonic	Format	Operation
—	0111	IN	R-Type variant	Rd = input_port
—	0111	OUT	R-Type variant	output_port = Rs

See Section 6.8 for full instruction definitions.

4.6 Option C: Register-Mapped I/O

Dedicate specific registers to I/O. Inputs appear in one register; outputs are driven from another.

Implementation:

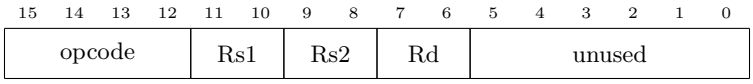
- **Input Register (read-only view):** Reading `R0` returns current input pin state
- **Output Register:** Writing `R2` drives output pins

5 Instruction Formats

All instructions are 16 bits wide. There are three instruction formats.

5.1 R-Type (Register Operations)

Used for ALU operations, comparisons, and register moves.



Fields:

- `opcode` [15:12] — Operation to perform
- `Rs1` [11:10] — First source register
- `Rs2` [9:8] — Second source register
- `Rd` [7:6] — Destination register
- `unused` [5:0] — Reserved (should be set to 0)

Assembly syntax: `MNEMONIC Rs1 Rs2 Rd`

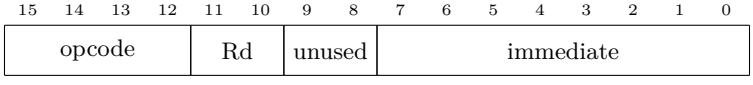
Example: `ADD R0 R1 R2` — Adds `R0` and `R1`, stores result in `R2`.

Example 2 — Adding a constant: To add a constant (e.g., 50) to `R0`, first load the constant into a register:

```
LDI R1 50       ; Load constant 50 into R1
ADD R0 R1 R0    ; R0 = R0 + 50
```

5.2 I-Type (Immediate)

Used for loading immediate values into registers.



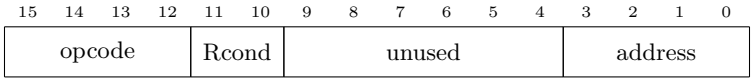
Fields:

- `opcode` [15:12] — Operation to perform
- `Rd` [11:10] — Destination register
- `unused` [9:8] — Reserved (should be set to 0)
- `immediate` [7:0] — 8-bit constant value (0–255 unsigned, or −128 to 127 signed)

Assembly syntax: `LDI Rd immediate`
Example: `LDI R0 42` — Loads the value 42 into `R0`.

5.3 J-Type (Jump)

Used for control flow (jumps and conditional branches).



Fields:
- `opcode` [15:12] — Jump type
- `Rcond` [11:10] — Register to test for conditional branches (JZ/JNZ). Ignored for JMP (don't-care, set to `00`).
- `unused` [9:4] — Reserved (should be set to 0)
- `address` [3:0] — Target instruction address (0–15)

Assembly syntax:
- `JMP address` — Unconditional jump
- `JZ Rcond address` — Jump if register Rcond = 0
- `JNZ Rcond address` — Jump if register Rcond ≠ 0

Examples:
- `JMP 5` — Jump to instruction at address 5
- `JNZ R3 5` — If `R3` is not zero, jump to address 5
- `JZ R0 3` — If `R0` is zero, jump to address 3

Design Note

The Rcond field allows conditional branches to test **any** register, not just `R3`. This makes programs shorter and more flexible — critical when the ROM is limited to 16 instructions. Comparison results still go to `R3`, but you can also branch directly on any register being zero/non-zero (e.g., a loop counter in `R0`).

6 Instruction Set Reference

6.1 Arithmetic Instructions

6.1.1 ADD — Add

Opcode	`0000`
Format	R-Type
Syntax	`ADD Rs1 Rs2 Rd`
Operation	`Rd = Rs1 + Rs2`
Description	Adds the values in Rs1 and Rs2, stores the result in Rd. Overflow wraps around (modulo 256).

```
LDI R0 10       ; R0 = 10
LDI R1 20       ; R1 = 20
ADD R0 R1 R2    ; R2 = 30
```

6.1.2 SUB — Subtract

Opcode	0001
Format	R-Type
Syntax	SUB Rs1 Rs2 Rd
Operation	Rd = Rs1 - Rs2
Description	Subtracts Rs2 from Rs1, stores the result in Rd. Uses two's complement for negative results.

```
LDI R0 50       ; R0 = 50
LDI R1 30       ; R1 = 30
SUB R0 R1 R2    ; R2 = 20
```

6.2 Bitwise Instructions

6.2.1 AND — Bitwise AND

Opcode	0010
Format	R-Type
Syntax	AND Rs1 Rs2 Rd
Operation	Rd = Rs1 & Rs2
Description	Performs bitwise AND on Rs1 and Rs2.

```
LDI R0 0b11110000   ; R0 = 240
LDI R1 0b10101010   ; R1 = 170
AND R0 R1 R2        ; R2 = 0b10100000 = 160
```

6.2.2 OR — Bitwise OR

Opcode	0011
Format	R-Type
Syntax	OR Rs1 Rs2 Rd
Operation	Rd = Rs1 | Rs2
Description	Performs bitwise OR on Rs1 and Rs2.

6.2.3 XOR — Bitwise Exclusive OR

Opcode	`0100`
Format	R-Type
Syntax	`XOR Rs1 Rs2 Rd`
Operation	`Rd = Rs1 ^ Rs2`
Description	Performs bitwise XOR on Rs1 and Rs2.

6.3 Shift Instructions

6.3.1 SHL — Shift Left

Opcode	`0101`
Format	R-Type
Syntax	`SHL Rs1 Rs2 Rd`
Operation	`Rd = Rs1 << Rs2`
Description	Shifts Rs1 left by Rs2 bits. Vacated bits are filled with zeros. Bits shifted out are discarded.

```
LDI R0 0b00000011   ; R0 = 3
LDI R1 2            ; R1 = 2 (shift amount)
SHL R0 R1 R2        ; R2 = 0b00001100 = 12
```

Note: Shifting left by 1 is equivalent to multiplying by 2.

6.3.2 SHR — Shift Right

Opcode	`0110`
Format	R-Type
Syntax	`SHR Rs1 Rs2 Rd`
Operation	`Rd = Rs1 >> Rs2`
Description	Shifts Rs1 right by Rs2 bits (logical shift). Vacated bits are filled with zeros.

Note: This is a logical shift (unsigned). Shifting right by 1 is equivalent to dividing by 2 (for unsigned values).

6.4 Data Movement Instructions

6.4.1 MOV — Move (Copy Register)

Opcode	`0111`
Format	R-Type
Syntax	`MOV Rs1 Rd`
Operation	`Rd = Rs1`
Description	Copies the value from Rs1 to Rd. Rs2 field is set to `00`.

```
LDI R0 99       ; R0 = 99
MOV R0 R2       ; R2 = 99
```

Encoding note: Rs2 bits [9:8] must be `00` to select MOV (vs IN/OUT in Option B).

6.4.2 LDI — Load Immediate

Opcode	`1010`
Format	I-Type
Syntax	`LDI Rd immediate`
Operation	`Rd = immediate`
Description	Loads an 8-bit constant value into register Rd.

```
LDI R0 255      ; R0 = 255 (max unsigned value)
LDI R1 -1       ; R1 = 255 (same in two's complement)
LDI R2 42       ; R2 = 42
```

6.5 Comparison Instructions

6.5.1 LT — Less Than

Opcode	`1000`
Format	R-Type
Syntax	`LT Rs1 Rs2`
Operation	`R3 = (Rs1 < Rs2) ? 1 : 0`
Description	Compares Rs1 and Rs2 as **signed** two's complement values. Sets `R3` to 1 if Rs1 < Rs2, otherwise 0. Rd field is ignored.

```
LDI R0 5        ; R0 = 5
LDI R1 10       ; R1 = 10
LT R0 R1        ; R3 = 1 (because 5 < 10)
JNZ R3 loop     ; Jump taken because R3 != 0
```

Note: To test "greater than", swap the operand order: `LT Rs2 Rs1` tests if Rs2 < Rs1, which is equivalent to Rs1 > Rs2.

Design Note

Signed comparison note: LT performs a signed comparison. Values in the range 1–100 (typical for the batch diverter project) are safe — they are positive in both signed and unsigned interpretation. Values loaded via LDI that exceed 127 are negative in signed two's complement (e.g., `LDI R0 200` loads the bit pattern for −56). This only matters if comparing mixed large values outside the normal project range.

6.5.2 EQ — Equal

Opcode	`1001`
Format	R-Type
Syntax	`EQ Rs1 Rs2`
Operation	`R3 = (Rs1 == Rs2) ? 1 : 0`
Description	Compares Rs1 and Rs2 for equality. Sets `R3` to 1 if equal, otherwise 0. Rd field is ignored.

```
LDI R0 42
LDI R1 42
EQ R0 R1        ; R3 = 1 (equal)
JNZ R3 match    ; Jump taken
```

6.6 Control Flow Instructions

6.6.1 JMP — Unconditional Jump

Opcode	1011
Format	J-Type
Syntax	`JMP address`
Operation	`PC = address`
Description	Sets the program counter to the specified address (0–15). Execution continues from that instruction. Rcond field is don't-care (set to 00).

```
JMP 10      ; Jump to instruction at address 10
```

6.6.2 JZ — Jump if Zero

Opcode	1100
Format	J-Type
Syntax	`JZ Rcond address`
Operation	`if (Rcond == 0) then PC = address`
Description	Jumps to the specified address if register Rcond is zero. Otherwise, continues to the next instruction. Rcond can be any register (R0–R3).

```
EQ R0 R1        ; R3 = 1 if equal, 0 if not
JZ R3 skip      ; Jump if NOT equal (R3 == 0)
; ... code if equal ...
; skip: ... continues here ...
```

6.6.3 JNZ — Jump if Not Zero

Opcode	1101
Format	J-Type
Syntax	`JNZ Rcond address`
Operation	`if (Rcond ≠ 0) then PC = address`
Description	Jumps to the specified address if register Rcond is not zero. Otherwise, continues to the next instruction. Rcond can be any register (R0–R3).

```
LT R0 R1        ; R3 = 1 if R0 < R1
JNZ R3 less     ; Jump if R0 < R1
; ... code if R0 >= R1 ...
; less: ... code if R0 < R1 ...
```

Branching on loop counters: Because Rcond can be any register, you can branch directly on a counter register without using a comparison instruction:

```
SUB R0 R1 R0    ; Decrement counter (R1 holds 1)
JNZ R0 loop     ; Loop while R0 != 0
```

6.7 Miscellaneous Instructions

6.7.1 Reserved — Opcode 1110

Opcode `1110` is **reserved** for team extensions. If your team implements a custom instruction (e.g., ADDI, NOT, INC, DEC), document it in your report. If not implemented, this opcode must trigger SYS_ERR as an illegal instruction (see Section 3.5).

Extension ideas:

- `ADDI Rd imm` — Add immediate to register
- `NOT Rs Rd` — Bitwise NOT
- `INC Rd` / `DEC Rd` — Increment/decrement
- `SWAPNIB Rd` — Swap nibbles (rotate by 4)

Note: If you need a no-operation, use `MOV R0 R0` (copies R0 to itself).

6.7.2 HALT — Halt Execution

Opcode	`1111`
Format	—
Syntax	`HALT`
Operation	Sets CPU_HALT latch
Description	Sets the internal CPU_HALT flag, which stops the program counter from advancing. No further instructions execute.

Encoding: `1111 0000 0000 0000` (all zeros after opcode)

Platform behaviour:

- HALT sets CPU_HALT → PC frozen, registers and output port preserved
- Platform treats CPU_HALT as a halt source: HALTED = STOP | FAULT | SYS_ERR | CPU_HALT
- MOTOR_EN is forced to 0 (motor safety interlock, see Section 3.6)
- ALARM is **not** asserted (HALT is intentional, not a fault)
- **Recovery:** Only REBOOT clears CPU_HALT and restarts execution from address 0

Note: In normal batch diverter operation, the program loops infinitely and HALT is never reached. HALT is useful for test programs and as a safety backstop.

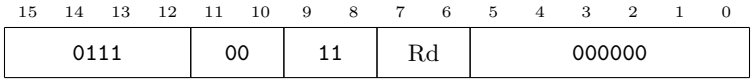
6.8 I/O Instructions (Option B)

If your team selects **Option B: Dedicated I/O Instructions**, implement the following. If using Option A or C, skip this section.

6.8.1 IN — Read Input Port

Opcode	Shares with MOV: `0111` with Rs2 = `11`
Format	R-Type (special)
Syntax	`IN Rd`
Operation	`Rd = input_port` (clears ITEM_PULSE latch)
Description	Reads the current state of the input port into register Rd. The ITEM_PULSE latch is cleared on read (see Section 4.2).

Encoding:



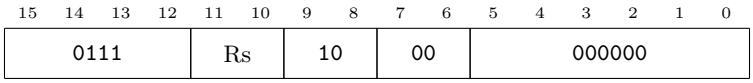
The `Rs2 = 11` distinguishes IN from MOV (where Rs2 = 00).

```
IN R0          ; R0 = input port state (bit 0 = ITEM_PULSE)
LDI R1 0x01    ; Mask for ITEM_PULSE (bit 0)
AND R0 R1 R0   ; R0 = 1 if item detected, else 0
```

6.8.2 OUT — Write Output Port

Opcode	Shares with MOV: `0111` with Rs2 = `10`
Format	R-Type (special)
Syntax	`OUT Rs`
Operation	`output_port = Rs`
Description	Writes the contents of Rs to the output port (GATE_SEL, MOTOR_EN, STATUS).

Encoding:



The `Rs2 = 10` distinguishes OUT from MOV and IN.

```
LDI R2 0x02    ; MOTOR_EN = 1, GATE_SEL = 0 (Gate A)
OUT R2         ; Write to output port
```

6.8.3 I/O Encoding Summary (Option B)

Rs2	Instruction	Operation
00	`MOV Rs1 Rd`	Rd = Rs1
01	(undefined — triggers SYS_ERR)	—
10	`OUT Rs1`	output_port = Rs1
11	`IN Rd`	Rd = input_port

Design Note

By overloading the MOV opcode with Rs2 variants, we avoid consuming additional opcodes while providing clean I/O semantics. The decoder checks Rs2 to select between register move and port operations. The `Rs2=01` encoding is undefined and must trigger SYS_ERR if executed (see Section 3.5).

7 Instruction Encoding Summary

Opcode	Binary	Mnemonic	Format	Operation
0	0000	ADD	R	Rd = Rs1 + Rs2
1	0001	SUB	R	Rd = Rs1 − Rs2
2	0010	AND	R	Rd = Rs1 & Rs2
3	0011	OR	R	Rd = Rs1 | Rs2
4	0100	XOR	R	Rd = Rs1 ⊕ Rs2
5	0101	SHL	R	Rd = Rs1 ≪ Rs2
6	0110	SHR	R	Rd = Rs1 ≫ Rs2
7	0111	MOV	R	Rd = Rs1 (Rs2=00)
7	0111	OUT	R	output = Rs1 (Rs2=10)†
7	0111	IN	R	Rd = input (Rs2=11)†
8	1000	LT	R	R3 = (Rs1 < Rs2)
9	1001	EQ	R	R3 = (Rs1 = Rs2)
10	1010	LDI	I	Rd = immediate
11	1011	JMP	J	PC = address
12	1100	JZ	J	if Rcond=0: PC = addr
13	1101	JNZ	J	if Rcond≠0: PC = addr
14	1110	—	—	Reserved (extension or SYS_ERR)
15	1111	HALT	—	Set CPU_HALT

† Option B only. See Section 6.8 for I/O instruction details.

8 Binary Encoding Examples

8.1 Example 1: ADD R0 R1 R2

```
ADD  R0   R1   R2   unused
0000  00   01   10   000000

Binary: 0000 0001 1000 0000
Hex:    0x0180
```

8.2 Example 2: LDI R0 42

```
LDI  R0   --   immediate(42)
1010  00   00   00101010

Binary: 1010 0000 0010 1010
Hex:    0xA02A
```

8.3 Example 3: JNZ R3 5

```
JNZ  Rcond(R3) unused   address(5)
1101    11     000000    0101

Binary: 1101 1100 0000 0101
Hex:    0xDC05
```

8.4 Example 4: JZ R0 3

```
JZ   Rcond(R0) unused   address(3)
1100    00     000000    0011

Binary: 1100 0000 0000 0011
Hex:    0xC003
```

8.5 Example 5: JMP 10

```
JMP  Rcond(--) unused   address(10)
1011    00     000000    1010

Binary: 1011 0000 0000 1010
Hex:    0xB00A
```

8.6 Example 6: LT R0 R1 (compare)

```
LT   R0   R1   --   unused
1000  00   01   00   000000

Binary: 1000 0001 0000 0000
Hex:    0x8100
```

8.7 Example 7: IN R0 (Option B)

```
IN   --   11   R0   unused
0111  00   11   00   000000

Binary: 0111 0011 0000 0000
Hex:    0x7300
```

8.8 Example 8: OUT R2 (Option B)

```
OUT  R2   10   --   unused
0111  10   10   00   000000

Binary: 0111 1010 0000 0000
Hex:    0x7A00
```

9 Program Memory

The EE8 uses a ROM (Read-Only Memory) to store programs.

Parameter	Value
Word size	16 bits (one instruction)
Address width	4 bits
Capacity	16 words
Address range	0–15 (`0x0`–`0xF`)

9.1 Program Counter (PC)

- 4-bit register
- Initialises to 0 on reset
- Increments by 1 after each instruction (unless jump taken)
- Wraps around from 15 to 0
- Frozen when HALTED (see Section 3.6)

9.2 Loading Programs

Students will build two ROM modules:

1. **External ROM** — Instructor-provided test programs (pre-loaded)
2. **Internal ROM** — Student-built program memory

A **LOAD signal** transfers the test program from external ROM to internal ROM before execution begins.

9.3 Fetch-Decode-Execute Cycle

1. **Fetch:** Read instruction from ROM at address PC
2. **Decode:** Extract opcode and operands
3. **Execute:** Perform operation
4. **Update PC:** PC = PC + 1 (or jump target if branch taken)

10 Timing and Clocking

The CPU operates on a single clock signal in a **single-cycle architecture**.

Mode	Description
Step mode	Manual clock button for debugging — one instruction per press
Run mode	Continuous clock signal from oscillator

10.1 Clock Phases (single-cycle)

On each rising clock edge:

1. Current instruction executes (fetch, decode, execute happen combinationally)
2. Results written to registers
3. PC updates

All operations are **atomic** — each instruction completes in a single clock cycle.

Design Note

Single-cycle architecture is simpler to design and understand than pipelined CPUs, making it ideal for educational purposes. The trade-off is that all instructions take the same time, even if some could be faster.

11 Reset State Summary

On REBOOT (or initial power-on), the CPU enters the following known state:

Element	Reset Value	Notes
R0	0x00	
R1	0x00	
R2	0x00	
R3	0x00	
PC	0x0	Execution starts at address 0
Output port	0x00	MOTOR_EN=0, GATE_SEL=0 (Gate A)
SYS_ERR	0	Cleared
CPU_HALT	0	Cleared

12 Assembly Language Syntax

12.1 Comments

```
; This is a comment
ADD R0 R1 R2    ; Inline comment
```

12.2 Labels

```
start:  LDI R0 10
        LDI R1 1
loop:   SUB R0 R1 R0
        JNZ R0 loop     ; Jump to 'loop' if R0 != 0
        HALT
```

12.3 Numeric Formats

```
LDI R0 42           ; Decimal
LDI R0 0x2A         ; Hexadecimal
LDI R0 0b00101010   ; Binary
```

13 Example Programs

13.1 Batch Diverter: $A \neq B$ with Full Alternation (Option B)

This is the primary example: a correct batch diverter that sends $A = 5$ items to Gate A, then $B = 3$ items to Gate B, repeating forever. Uses all 16 instruction slots.

```
; Batch diverter: A=5 to Gate A, B=3 to Gate B, alternating forever
; Option B I/O. All 16 instruction slots used.
;
; Register usage:
;   R0 = item counter (counts down to 0)
;   R1 = scratch (input, constants)
;   R2 = output port value (bit 0 = GATE_SEL, bit 1 = MOTOR_EN)
;   R3 = scratch (masks)
;
; After REBOOT: all regs = 0, PC = 0.

; === Setup (runs once at start, also re-executed on Gate A reload) ===
0:  LDI R0 5            ; R0 = batch count A (5 items)
1:  LDI R2 0x02         ; R2 = MOTOR_EN=1, GATE_SEL=0 (Gate A)

; === Counting loop (shared for both gates) ===
2:  OUT R2              ; Drive output: motor on, current gate
3:  IN R1               ; Read input port (clears ITEM_PULSE latch)
4:  LDI R3 0x01         ; ITEM_PULSE mask (bit 0)
5:  AND R1 R3 R1        ; Isolate ITEM_PULSE
6:  JZ R1 3             ; No item detected? Keep polling
7:  LDI R1 1            ; Decrement constant
8:  SUB R0 R1 R0        ; R0 = R0 - 1
9:  JNZ R0 3            ; More items in batch? Continue counting

; === Batch complete: toggle gate and reload count ===
10: LDI R1 0x01         ; GATE_SEL toggle mask (bit 0)
11: XOR R2 R1 R2        ; Flip GATE_SEL: Gate A (0x02) <-> Gate B (0x03)
12: AND R2 R1 R3        ; R3 = new GATE_SEL (0 = Gate A, 1 = Gate B)
13: LDI R0 3            ; Load count B (optimistic for Gate B)
14: JNZ R3 2            ; If Gate B selected (R3=1) -> counting loop
15: LDI R0 5            ; Gate A selected: load count A = 5
                         ; PC wraps 15 -> 0
```

Execution trace (first 3 batches):

Phase	Addresses	R0	R2	Gate	Items
Setup	$0 \to 1$	5	0x02	A	—
Batch 1	$2 \to \cdots \to 9$	$5 \to 0$	0x02	A	5
Toggle	$10 \to \cdots \to 14 \to 2$	3	0x03	B	—
Batch 2	$2 \to \cdots \to 9$	$3 \to 0$	0x03	B	3
Toggle	$10 \to \cdots \to 15 \to 0 \to 1 \to 2$	5	0x02	A	—
Batch 3	$2 \to \cdots \to 9$	$5 \to 0$	0x02	A	5
...	repeats				

Pattern: 5(A), 3(B), 5(A), 3(B), ... ✓

13.2 Simple Batch Diverter: Equal Counts (Option B)

A simpler version where both gates get the same count. Useful as a starting point.

```
; Simple batch diverter: 5 items per gate, alternating
; Option B I/O. 12 instructions.

; === Setup ===
0:  LDI R2 0x02          ; MOTOR_EN=1, GATE_SEL=0 (Gate A)
1:  LDI R0 5             ; Batch count = 5

; === Counting loop ===
2:  OUT R2               ; Drive outputs
3:  IN R1                ; Read input
4:  LDI R3 0x01          ; ITEM_PULSE mask
5:  AND R1 R3 R1         ; Isolate pulse
6:  JZ R1 3              ; Poll until item arrives
7:  LDI R1 1             ; Decrement constant
8:  SUB R0 R1 R0         ; R0--
9:  JNZ R0 3             ; Count more items

; === Toggle and restart ===
10: LDI R1 0x01          ; Toggle mask
11: XOR R2 R1 R2         ; Flip GATE_SEL
                          ; PC falls through to 12
12: LDI R0 5             ; Reload same count
13: JMP 2                ; Back to counting loop

14: HALT                 ; Never reached
15: HALT
```

13.3 Arithmetic Example: Add Two Numbers

```
; Adds 25 + 17, stores result in R2
; 4 instructions
0:  LDI R0 25        ; R0 = 25
1:  LDI R1 17        ; R1 = 17
2:  ADD R0 R1 R2     ; R2 = 42
3:  HALT
```

13.4 Find Maximum of Two Numbers

```
; Finds max of R0 and R1, stores in R2
; 8 instructions
0:  LDI R0 45            ; R0 = 45
1:  LDI R1 72            ; R1 = 72
2:  LT R0 R1             ; R3 = 1 if R0 < R1
3:  JNZ R3 6             ; If R0 < R1, jump to r1_bigger
4:  MOV R0 R2            ; R0 is bigger or equal
5:  JMP 7                ; Skip to done
6:  MOV R1 R2            ; R1 is bigger
7:  HALT                 ; R2 = 72 (the maximum)
```

13.5 Polling Input Example (Option B)

```
; Wait for ITEM_PULSE, then turn off motor
; Demonstrates I/O polling pattern
; 7 instructions

0:  LDI R2 0x02          ; MOTOR_EN=1, GATE_SEL=0
1:  OUT R2               ; Enable motor
2:  IN R0                ; Read inputs
3:  LDI R1 0x01          ; ITEM_PULSE mask
4:  AND R0 R1 R0         ; Isolate bit 0
5:  JZ R0 2              ; No pulse? Keep polling
6:  HALT                 ; Item detected, stop
```

13.6 Countdown Example

```
; Counts down from 10 to 0 using the JNZ Rcond syntax
; Demonstrates branching on a counter register directly
; 4 instructions

0:  LDI R0 10            ; R0 = counter = 10
1:  LDI R1 1             ; R1 = decrement constant
2:  SUB R0 R1 R0         ; R0 = R0 - 1
3:  JNZ R0 2             ; Loop while R0 != 0
4:  HALT                 ; R0 = 0, done
```

A Quick Reference Card

EE8 CPU Quick Reference — Version 4.1

Registers

R0, R1	General purpose
R2	General purpose
R3	General purpose + comparison dest

Arithmetic

```
ADD Rs1 Rs2 Rd
SUB Rs1 Rs2 Rd
```

Bitwise

```
AND Rs1 Rs2 Rd
OR  Rs1 Rs2 Rd
XOR Rs1 Rs2 Rd
```

Shift

```
SHL Rs1 Rs2 Rd
SHR Rs1 Rs2 Rd
```

Compare (result → R3)

LT Rs1 Rs2	(R3 = Rs1<Rs2)
EQ Rs1 Rs2	(R3 = Rs1=Rs2)

Data Movement

MOV Rs Rd	Copy register
LDI Rd imm	Load immediate (0–255)
IN Rd	Read input port (Opt. B)
OUT Rs	Write output port (Opt. B)

Control Flow (addr: 0–15)

JMP addr	Unconditional jump
JZ Rcond addr	Jump if Rcond = 0
JNZ Rcond addr	Jump if Rcond ≠ 0

Miscellaneous

HALT	Stop execution
(opcode 1110)	Reserved / SYS_ERR

I/O Port Bits

Input [0]	ITEM_PULSE (cleared on read)
Output [0]	GATE_SEL
Output [1]	MOTOR_EN
Output [2+]	STATUS (optional)

Platform Signals (hardware, NOT software-polled)

REBOOT	Async reset (clears all state, PC=0)
STOP	Pause (clock gated, state preserved)
FAULT	Pause + ALARM
SYS_ERR	Illegal insn → halt + ALARM (latched)
CPU_HALT	HALT insn → halt, no ALARM (latched)

$\text{HALTED} = \text{STOP} \mid \text{FAULT} \mid \text{SYS\_ERR} \mid \text{CPU\_HALT}$
$\text{MOTOR\_EN}_{\text{pin}} = \text{MOTOR\_EN}_{\text{cpu}} \wedge \overline{\text{HALTED}}$
$\text{ALARM} = \text{FAULT} \mid \text{SYS\_ERR}$

Memory: 16 instructions (addresses 0–F). PC wraps from 15 → 0.

B Opcode Map

Binary	Mnemonic	Binary	Mnemonic
0000	ADD	1000	LT
0001	SUB	1001	EQ
0010	AND	1010	LDI
0011	OR	1011	JMP
0100	XOR	1100	JZ
0101	SHL	1101	JNZ
0110	SHR	1110	(reserved / SYS_ERR)
0111	MOV/IN/OUT	1111	HALT

MOV/IN/OUT disambiguation (opcode 0111):

Rs2	Instruction
00	`MOV Rs1 Rd`
01	(undefined → SYS_ERR)
10	`OUT Rs1`
11	`IN Rd`

J-Type format (JMP/JZ/JNZ):

```
[15:12] opcode   [11:10] Rcond   [9:4] unused   [3:0] address
```

- JMP: Rcond ignored (set to 00)
- JZ: jump if Rcond = 0
- JNZ: jump if Rcond ≠ 0

C I/O Interface Options Summary

Option	Mechanism	Pros	Cons
A	Memory-mapped	No new instructions	Loses ROM addresses
B	IN/OUT instructions	Clean, explicit	Uses opcode space
C	Register-mapped	Simplest hardware	Loses registers

I/O Port Signals (all options):

Port	Bit	Signal	Notes
Input	0	ITEM_PULSE	Cleared on read
Output	0	GATE_SEL	0 = Gate A, 1 = Gate B
Output	1	MOTOR_EN	Request (overridden when HALTED)
Output	2+	STATUS	Optional debug bits

Not in I/O ports (hardware-driven): ALARM, STOP, FAULT, REBOOT

D Changes from Previous Versions

Version 4.1 (correction)

- **I-Type encoding fixed**: Rd is at bits [11:10], not [9:8]. Bytefield diagram, field list, and Example 2 corrected to match reference CPU implementation and Count-to-5 handout.

Version 4.0 (platform signals, Rcond branches, safety)

- **New Section 3: Platform Signals & Safety Logic** — REBOOT, STOP, FAULT, SYS_ERR, CPU_HALT fully specified as hardware signals, not software-polled I/O
- **J-Type format changed**: added Rcond field [11:10] — JZ/JNZ can now test any register, not just `R3`
- **HALT behaviour clarified**: sets CPU_HALT latch, MOTOR_EN forced 0, ALARM not asserted, only cleared by REBOOT
- **SYS_ERR fully specified**: triggers on undefined opcode or undefined Rs2=01 sub-encoding, latched until REBOOT

- **Platform halt logic**: HALTED = STOP | FAULT | SYS_ERR | CPU_HALT
- **Alarm logic**: ALARM = FAULT | SYS_ERR (hardware-driven, removed from CPU output port)
- **I/O port revised**: input carries only ITEM_PULSE (bit 0), STOP/FAULT/REBOOT removed from input port
- **ITEM_PULSE semantics**: hardware-latched, cleared on read (prevents multi-counting)
- **Reset state specified**: Section 11 documents all state after REBOOT
- **Signed comparison note** added for LT instruction
- **Section numbering fixed** throughout
- **All examples corrected**: fixed register name typos, fixed polling/counting loop bugs, all examples use new JZ/JNZ Rcond syntax
- **New example 13.1**: correct $A \neq B$ batch diverter with full execution trace

Version 3.0 (I/O and batch diverter)

- Added I/O interface section (Options A, B, C)
- Added IN and OUT instructions (Option B) via MOV opcode overloading
- Renamed R0 → R2, RF → R3 for uniformity (all registers now general-purpose)
- Removed NOP (opcode 1110 now reserved for team extensions)
- Updated examples for batch diverter scenario
- Added I/O port bit assignments for station signals
- Aligned with EG2300 project specification

Version 2.0 (4-bit addressing)

- Reduced program memory from 256 to 16 instructions
- Changed PC from 8-bit to 4-bit counter
- Updated J-type format: address field now [3:0] (4 bits)
- All jump/branch addresses now 0–15 range
- Added design rationale for comparison register approach

End of EE8 CPU ISA Specification